FAST TCP: Motivation, Architecture, Algorithms, Performance

David X. Wei, *Student Member, IEEE*, Cheng Jin, Steven H. Low, *Senior Member, IEEE*, and Sanjay Hegde

***Abstract*—We describe FAST TCP, a new TCP congestion control algorithm for high-speed long-latency networks, from design to implementation. We highlight the approach taken by FAST TCP to address the four difficulties which the current TCP implementation has at large windows. We describe the architecture and summarize some of the algorithms implemented in our prototype. We characterize its equilibrium and stability properties. We evaluate it experimentally in terms of throughput, fairness, stability, and responsiveness.**

***Index Terms*—FAST TCP, implementation, Internet congestion control, protocol design, stability analysis.**

I. Introduction and Summary

CONGESTION control is a distributed algorithm to share network resources among competing users. It is important in situations where the availability of resources and the set of competing users vary over time unpredictably, yet efficient and fair sharing is desired. These constraints—unpredictable supply and demand and the desire for efficient distributed operation—necessarily lead to feedback control as the preferred approach, where traffic sources dynamically adapt their rates to congestion in their paths. On the Internet, this is performed by the Transmission Control Protocol (TCP) in source and destination computers involved in data transfers.

The congestion control algorithm in the current TCP, which we refer to as Reno in this paper, was developed in 1988 [20] and has gone through several enhancements since, e.g., [1], [14], [18], [21], [47], [58]. It has performed remarkably well and is generally believed to have prevented severe congestion as the Internet scaled up by six orders of magnitude in size, speed, load, and connectivity. It is also well-known, however, that as bandwidth–delay product continues to grow, TCP Reno will eventually become a performance bottleneck itself. The following four difficulties contribute to the poor performance of TCP Reno in networks with large bandwidth–delay products.

1) At the packet level, linear increase by one packet per round-trip time (RTT) is too slow, and multiplicative decrease per loss event is too drastic.
2) At the flow level, maintaining large average congestion windows *requires* an extremely small equilibrium loss probability.
3) At the packet level, oscillation in congestion window is unavoidable because TCP uses a binary congestion signal (packet loss).
4) At the flow level, the dynamics is unstable, leading to severe oscillations that can only be reduced by the accurate estimation of packet loss probability and a stable design of the flow dynamics.

We explain these difficulties in Section II and motivate a delay-based solution. Delay-based congestion control has been proposed, e.g., in [3], [12], [23], [69], [72]. See [4], [5], [27], [28], [36], [56], [68], [75] for other recent proposals.

Using queueing delay as a congestion measure has two advantages. First, queueing delay can be more accurately estimated than loss probability both because packet losses in networks with large bandwidth–delay product need to be rare events under TCP Reno (e.g., probability on the order 10^{-7} or smaller), and because loss samples provide coarser information than queueing delay samples. Indeed, measurements of delay are noisy, just as those of loss probability. Each measurement of packet loss (whether a packet is lost) provides one bit of information for the filtering of noise, whereas each measurement of queueing delay provides multi-bit information. This makes it easier for an equation-based implementation to stabilize a network into a steady state with a target fairness and high utilization. Second, based on the commonly used ordinary differential equation model of TCP/AQM, the dynamics of queueing delay has the right scaling with respect to network capacity. This helps maintain stability as a network scales up in capacity [8], [51], [53].

In Section III, we lay out an architecture and present an overview of some of the algorithms implemented in our first prototype. Even though the discussion is in the context of FAST TCP, the architecture can serve as a general framework to guide the design of other congestion control mechanisms, not necessarily limited to TCP, for high-speed networks. The main components in the architecture can be designed separately and upgraded asynchronously.

We evaluate FAST TCP both analytically and experimentally. In Section III-B, we present a mathematical model of the window control algorithm. We prove that FAST TCP has the same equilibrium properties as TCP Vegas [44], [50]. In particular, it does not penalize flows with large propagation delays and it achieves weighted proportional fairness [31]. For the special case of single bottleneck link with heterogeneous flows, we prove that the window control algorithm of FAST is locally asymptotically stable, in the absence of feedback delay.

In Section IV, we present both experimental and simulation results to illustrate throughput, fairness, stability, and responsiveness of FAST TCP, in the presence of delay and in hetero-

Manuscript received November 28, 2004; revised September 14, 2005; approved by IEEE/ACM Transactions on Networking Editor E. Knightly. This work was supported by the NSF under Grants ANI-0113425 and ANI-0230967, Caltech Lee Center for Advanced Networking, the ARO under Grant DAAD19-02-1-0283, the AFOSR under Grant F49620-03-1-0119, DARPA, and Cisco. An abridged version of this paper appears in Proceedings of IEEE INFOCOM, March 2004, and an expanded version appears online at http://netlab.caltech.edu/FAST.

D. X. Wei, C. Jin, and S. H. Low are with the Division of Engineering and Applied Science, California Institute of Technology, Pasadena, CA 91125 USA (e-mail: slow@caltech.edu).

S. Hegde is with Akamai Technologies, San Mateo, CA 94404 USA.

Digital Object Identifier 10.1109/TNET.2006.886335

1063-6692/$20.00 © 2006 IEEE
Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

geneous and dynamic environments where flows of different delays join and depart asynchronously. It is important to evaluate a congestion control algorithm not only in terms of throughput achieved, but also what it does to network queues and how that affects other applications sharing the same queue. We compare the performance of FAST TCP with Reno, HSTCP [15], STCP [32], and BIC TCP [75], using their default parameters.

In Section V, we summarize open issues and provide references for proposed solutions.

II. Motivations

A congestion control algorithm can be designed at two levels. The *flow*-level (macroscopic) design aims to achieve high utilization, low queueing delay and loss, fairness, and stability. The *packet*-level design implements these flow level goals within the constraints imposed by end-to-end control. Historically for TCP Reno, packet-level implementation was introduced first. The resulting flow-level properties, such as fairness, stability, and the relationship between equilibrium window and loss probability, were then understood as an afterthought. In contrast, the packet-level designs of HSTCP [15], STCP [32], and FAST TCP are explicitly guided by flow-level goals.

A. Packet and Flow Level Modeling

The congestion avoidance algorithm of TCP Reno and its variants have the form of AIMD [20]. The pseudo code for window adjustment is

$$\begin{aligned} \texttt{Ack:} \quad & \texttt{w} \longleftarrow \texttt{w} + \frac{1}{\texttt{w}} \\ \texttt{Loss:} \quad & \texttt{w} \longleftarrow \texttt{w} - \frac{1}{2}\texttt{w} \end{aligned}$$

This is a packet-level model, but it induces certain flow-level properties such as throughput, fairness, and stability.

These properties can be understood with a flow-level model of the AIMD algorithm, e.g., [19], [29], [39], [41]. The window $w_i(t)$ of source i increases by 1 packet per RTT,[1] and decreases per unit time by

$$x_i(t)q_i(t) \cdot \frac{1}{2} \cdot \frac{4}{3} w_i(t) \quad \text{packets}$$

where $x_i(t) := w_i(t)/T_i(t)$ pkts/s. $T_i(t)$ is the round-trip time, and $q_i(t)$ is the (delayed) end-to-end loss probability, in period t.[2] Here, $4w_i(t)/3$ is the peak window size that gives the "average" window of $w_i(t)$. Hence, a flow-level model of AIMD is

$$\dot{w}_i(t) = \frac{1}{T_i(t)} - \frac{2}{3} x_i(t) q_i(t) w_i(t). \tag{1}$$

Setting $\dot{w}_i(t) = 0$ in (1) yields the well-known $1/\sqrt{q}$ formula for TCP Reno discovered in [37], [48], which relates loss probability to window size in equilibrium:

$$q_i^* = \frac{3}{2w_i^{*2}}. \tag{2}$$

[1]It should be $(1 - q_i(t))$ packets, where $q_i(t)$ is the end-to-end loss probability. This is roughly 1 when $q_i(t)$ is small.

[2]This model assumes that window is halved on each packet loss. It can be modified to model the case, where window is halved at most once in each RTT. This does not qualitatively change the following discussion.

TABLE I
Common Dynamic Structure: w_i is Source i's Window Size, T_i is Its Round-Trip Time, q_i is Congestion Measure, $x_i = w_i/T_i$; a, $b(w_i)$, ρ, γ, α_i, τ are Protocol Parameters; See [24]

	$\kappa_i(w_i, T_i)$	$u_i(w_i, T_i)$	q_i
FAST	$\gamma\alpha_i/\tau$	α_i/x_i	queueing delay
Reno	$1/T_i$	$1.5/w_i^2$	loss probability
HSTCP	$\frac{0.16b(w_i)w_i^{0.80}}{(2-b(w_i))T_i}$	$0.08/w_i^{1.20}$	loss probability
STCP	aw_i/T_i	ρ/w_i	loss probability

TABLE II
Common Equilibrium Structure: x_i is Source i's Throughput in packets/sec, T_i is Equilibrium Round-Trip Time, q_i is End-to-End Congestion Measure in Equilibrium. The Parameters are: $\alpha = 1.225$ for Reno, $\alpha = 0.120$ for HSTCP, and $\alpha = 0.075$ for STCP. For FAST, α_i Should Vary With Link Capacity

FAST	$x_i = \frac{\alpha_i}{q_i}$
Reno	$x_i = \frac{1}{T_i} \cdot \frac{\alpha_i}{q_i^{0.50}}$
HSTCP	$x_i = \frac{1}{T_i} \cdot \frac{\alpha_i}{q_i^{0.84}}$
STCP	$x_i = \frac{1}{T_i} \cdot \frac{\alpha_i}{q_i}$

In summary, (1) and (2) describe the flow-level dynamics and equilibrium, respectively, for TCP Reno.

Even though Reno, HSTCP, STCP, and FAST look different at the packet level, they have similar equilibrium and dynamic structures at the flow level; see [24] for detailed derivations. The congestion windows in these algorithms all evolve according to

$$\dot{w}_i(t) = \kappa_i(t) \cdot \left(1 - \frac{q_i(t)}{u_i(t)}\right) \tag{3}$$

where $\kappa_i(t) := \kappa_i(w_i(t), T_i(t))$ and $u_i(t) := u_i(w_i(t), T_i(t))$. They differ only in the choice of the gain function $\kappa_i(w_i, T_i)$, the marginal utility function $u_i(w_i, T_i)$, and the end-to-end congestion measure q_i. Within this structure, at the flow level, there are thus only three design decisions:

- $\kappa_i(w_i, T_i)$: the choice of the gain function κ_i determines the dynamic properties such as stability and responsiveness, but does not affect the equilibrium properties;
- $u_i(w_i, T_i)$: the choice of the marginal utility function u_i determines equilibrium properties such as the equilibrium rate allocation and its fairness;
- q_i: in the absence of explicit feedback, the choice of congestion measure q_i is limited to loss probability or queueing delay. The dynamics of $q_i(t)$ is determined inside the network.

At the flow level, a goal is to design a class of function pairs, $u_i(w_i, T_i)$ and $\kappa_i(w_i, T_i)$, so that the feedback system described by (3), together with link dynamics of $q_i(t)$ and the interconnection, has an equilibrium that is fair and efficient, and that the equilibrium is stable, in the presence of feedback delay. The design choices in FAST, Reno, HSTCP, and STCP are shown in Table I. These choices produce equilibrium characterizations shown in Table II.

We next illustrate the equilibrium and dynamics problems of TCP Reno, at both the packet and flow levels, as bandwidth–delay product increases.

B. Reno's Problems at Large Window

The equilibrium problem at the flow level is expressed in (2): the end-to-end loss probability must be exceedingly small to

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

sustain a large window size, making the equilibrium difficult to maintain in practice, as bandwidth–delay product increases. Indeed, from (2), $q_i^* w_i^* = 1.5/w_i^*$, i.e., the average number of packet losses (or loss events) per window decreases in inverse proportion to the equilibrium window size for Reno. From Table II, this number for HSTCP is $q_i^* w_i^* = 0.0789/(w_i^*)^{0.20}$. Hence, it also decreases with the equilibrium window, but more slowly than for TCP Reno. For STCP, this number is $q_i^* w_i^* = a(1-b/2)/b$, which is independent of, and hence scalable with, the equilibrium window size. The recommended values in [32] for the constants are $a = 0.01$ and $b = 0.125$, yielding an average loss of 0.075 per window. Even though equilibrium is a flow-level notion, this problem with Reno manifests itself at the packet level, where a source increases its window too slowly and decreases it too drastically. In contrast, HSTCP and STCP increase more aggressively and decrease less drastically.

The causes of the oscillatory behavior of TCP Reno lie in its design at both the packet and flow levels. At the packet level, the choice of binary congestion signal necessarily leads to oscillation in congestion windows and bottleneck queues, and the parameter setting in Reno worsens the situation as bandwidth–delay product increases. At the flow level, the system dynamics given by (1) is unstable at large bandwidth–delay products [19], [39]. These must be addressed by different means.

Congestion window can be stabilized only if multi-bit feedback is used. This is the approach taken by the equation-based algorithm in [13], where congestion window is adjusted based on the estimated loss probability in an attempt to stabilize around a target value given by (2). This approach eliminates the oscillation due to packet-level AIMD, but two difficulties remain at the flow level.

First, equation-based control requires the explicit estimation of end-to-end loss probability. This is difficult when the loss probability is small. Second, even if loss probability can be perfectly estimated, Reno's flow dynamics, described by equation (1) leads to a feedback system that becomes unstable as feedback delay increases, and more strikingly, as network capacity increases [19], [39]. The instability at the flow level can lead to severe oscillations that can be reduced *only* by stabilizing the flow level dynamics. We present a delay-based approach to address these problems.

C. Delay-Based Approach

The common model (3) can be interpreted as follows: the goal at the flow level is to equalize marginal utility $u_i(t)$ with the end-to-end measure of congestion $q_i(t)$. This interpretation immediately suggests an equation-based packet-level implementation where the window adjustment $\dot{w}_i(t)$ depends on not only the sign, but also the magnitude of the difference between the ratio $q_i(t)/u_i(t)$ and the target of 1. Unlike the approach taken by Reno, HSTCP, and STCP, this approach eliminates packet-level oscillations due to the binary nature of congestion signal. It however requires the *explicit* estimation of the end-to-end congestion measure $q_i(t)$.

Without explicit feedback, $q_i(t)$ can only be loss probability, as used in TFRC [13], or queueing delay, as used in TCP Vegas [3] and FAST TCP. Queueing delay can be more accurately estimated than loss probability both because loss samples provide coarser information than queueing delay samples, and because packet losses in networks with large bandwidth–delay products need to be rare events under schemes such as Reno. Indeed, each measurement of packet loss (whether a packet is lost) provides one bit of information for the filtering of noise, whereas each measurement of queueing delay provides multi-bit information. This facilitates an equation-based implementation to stabilize a network into a steady state with a target fairness and high utilization.

At the flow level, the dynamics of the feedback system must be stable in the presence of delay, as the network capacity increases. Here, again, queueing delay has an advantage over loss probability as a congestion measure: the dynamics of queueing delay have the right scaling with respect to network capacity, according to the commonly used ordinary differential equation model. This helps maintain stability as network capacity grows [8], [51]–[53].[3]

This motivates the following implementation strategy. First, by explicitly estimating how far the current state $q_i(t)/u_i(t)$ is from the equilibrium value of 1, a delay-based scheme can drive the system rapidly, yet in a fair and stable manner, toward the equilibrium. The window adjustment is small when the current state is close to equilibrium and large otherwise, *independent of where the equilibrium is*. This is in stark contrast to the approach taken by Reno, HSTCP, and STCP, where window adjustment depends on just the current window size and is independent of where the current state is with respect to the target (compare [24, Figs. 1(a) and (b)]). Like the equation-based scheme in [13], this approach avoids the problem of slow increase and drastic decrease in Reno, as the network scales up. Second, by choosing a multi-bit congestion measure, this approach eliminates the packet-level oscillation due to binary feedback, avoiding Reno's third problem. Third, using queueing delay as the congestion measure $q_i(t)$ allows the network to stabilize in the region below the overflowing point, when the buffer size is sufficiently large. Stabilization at this operating point eliminates large queueing delay and unnecessary packet loss. More importantly, it makes room for buffering "mice" traffic. To avoid the second problem in Reno, where the required equilibrium congestion measure (loss probability for Reno, and queueing delay here) is too small to practically estimate, the algorithm must adapt its parameter α_i to capacity to maintain small but sufficient queueing delay. Finally, to avoid the fourth problem of Reno, the window control algorithm must be stable, in addition to being fair and efficient, at the flow level. The emerging theory of large-scale networks under end-to-end control, e.g., [6], [8], [11], [19], [31], [33]–[35], [39], [41], [42], [44], [46], [50]–[53], [56], [63], [64], [73], [76] (see also, e.g., [30], [40], [43], [57] for recent surveys), forms the foundation of the flow-level design. The theory plays an important role by providing a framework to understand issues, clarify ideas, and suggest directions, leading to a robust and high performance implementation.

III. Architecture and Algorithms

A. Architecture

We separate the congestion control mechanism of TCP into four components in Fig. 1. These four components are function-

[3]See [45] for the pitfalls of using delay to predict loss in the hope of helping a loss-based algorithm adjust its window.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

ally independent so that they can be designed separately and upgraded asynchronously.

The *data control* component determines *which* packets to transmit, *window control* determines *how many* packets to transmit, and *burstiness control* determines *when* to transmit these packets [24], [71]. These decisions are made based on information provided by the *estimation* component.

An initial prototype that included some of these features was demonstrated in November 2002 at the SuperComputing Conference, and the experimental results were reported in [26]. In the following, we explain in detail the design of the window control component.

B. Window Control Algorithm

FAST reacts to both queueing delay and packet loss. Under normal network conditions, FAST periodically updates the congestion window based on the average RTT according to

$$\mathtt{w} \quad \longleftarrow \quad \min\left\{2\mathtt{w},\ (1-\gamma)\mathtt{w}+\gamma\left(\frac{\mathtt{baseRTT}}{\mathtt{RTT}}\mathtt{w}+\alpha\right)\right\}$$

where $\gamma \in (0,1]$, baseRTT is the minimum RTT observed so far, and α is a positive protocol parameter that determines the total number of packets queued in routers in equilibrium along the flow's path. The window update period is 20 ms in our prototype.

We now provide an analytical evaluation of FAST TCP. We present a model of the window control algorithm for a network of FAST flows. We show that, in equilibrium, the vectors of source windows and link queueing delays are the unique solutions of a pair of optimization problems (6)–(7). This completely characterizes the network equilibrium properties such as throughput, fairness, and delay. We also present a preliminary stability analysis.

We model a network as a set of resources with finite capacities c_l, e.g., transmission links, processing units, memory, etc., to which we refer to as "links" in our model. The network is shared by a set of unicast flows, identified by their sources. Let d_i denote the round-trip propagation delay of source i. Let R be the routing matrix where $R_{li} = 1$ if source i uses link l, and 0 otherwise. Let $p_l(t)$ denote the queueing delay at link l at time t. Let $q_i(t) = \sum_l R_{li}p_l(t)$ be the round-trip queueing delay, or in vector notation, $q(t) = R^T p(t)$. Then the round-trip time of source i is $T_i(t) := d_i + q_i(t)$.

Each source i adapts its window $w_i(t)$ periodically according to[4]

$$w_i(t+1) = \gamma\left(\frac{d_i w_i(t)}{d_i + q_i(t)} + \alpha_i\right) + (1-\gamma)w_i(t) \qquad (4)$$

where $\gamma \in (0,1]$, at time t.

A key departure of our model from those in the literature is that we assume that a source's *send rate*, defined as $x_i(t) := w_i(t)/T_i(t)$, cannot exceed the *throughput* it receives. This is justified because of self-clocking: within one round-trip time after a congestion window is increased, packet transmission will be clocked at the same rate as the throughput the flow receives. See [66] for detailed justification and validation experiments. A consequence of this assumption is that the link queueing delay vector, $p(t)$, is determined implicitly by the instantaneous window size in a *static* manner: given $w_i(t) = w_i$ for all i, the link queueing delays $p_l(t) = p_l \geq 0$ for all l are given by

$$\sum_i R_{li}\frac{w_i}{d_i + q_i(t)} \begin{cases} = c_l & \text{if } p_l(t) > 0 \\ \leq c_l & \text{if } p_l(t) = 0 \end{cases} \qquad (5)$$

where again $q_i(t) = \sum_l R_{li}p_l(t)$.

The next result says that the queueing delay is indeed well defined. All proofs are relegated to the Appendix and [24].

Lemma 1: Suppose the routing matrix R has full row rank. Given $w = (w_i, \forall i)$, there exists a unique queueing delay vector $p = (p_l, \forall l)$ that satisfies (5).

The equilibrium values of windows w^* and delays p^* of the network defined by (4)–(5) can be characterized as follows. Consider the utility maximization problem

$$\max_{x \geq 0} \sum_i \alpha_i \log x_i \quad \text{s.t.} \quad Rx \leq c \qquad (6)$$

and the following (dual) problem:

$$\min_{p \geq 0} \quad \sum_l c_l p_l - \sum_i \alpha_i \log \sum_l R_{li}p_l. \qquad (7)$$

Theorem 2: Suppose R has full row rank. The unique equilibrium point (w^*, p^*) of the network defined by (4), (5) exists and is such that $x^* = (x_i^* := w_i/(d_i + q_i^*), \forall i)$ is the unique maximizer of (6) and p^* is the unique minimizer of (7). This implies in particular that the equilibrium rate x^* is α_i-weighted proportionally fair.

Theorem 2 implies that FAST TCP has the same equilibrium properties as TCP Vegas [44], [50]. Its equilibrium throughput is given by

$$x_i = \frac{\alpha_i}{q_i}. \qquad (8)$$

In particular, it does not penalize sources with large propagation delays d_i. The relation (8) also implies that, in equilibrium, source i maintains α_i packets in the buffers along its path [44], [50]. Hence, the total amount of buffering in the network must be at least $\sum_i \alpha_i$ packets in order to reach the equilibrium.[5]

We now turn to the stability of the algorithm.

Theorem 3 (Single-Link Heterogeneous-Source): Suppose there is only a single link with capacity c. Then the system defined by (4)–(5) is locally asymptotically stable.

The basic idea of the proof is to show that the mapping from (scaled) $w(t)$ to $w(t+1)$ defined by (4)–(5) has a Jacobian whose spectral radius is strictly less than 1, uniformly in w; see Theorem 6 in the Appendix. Hence, $w(t)$ converges locally to the unique equilibrium. The proof technique seems to

[4]Note that (4) can be rewritten as (when $\alpha_i(w_i, q_i) = \alpha_i$, constant)

$$w_i(t+1) = w_i(t) + \gamma_i\left(\alpha_i - x_i(t)q_i(t)\right).$$

From [44], TCP Vegas updates its window according to

$$w_i(t+1) = w_i(t) + \frac{1}{T_i(t)}\operatorname{sgn}\left(\alpha_i - x_i(t)q_i(t)\right)$$

where $\operatorname{sgn}(z) = -1$ if $z < 0$, 0 if $z = 0$, and 1 if $z > 0$. Hence, FAST can be thought of as a high-speed version of Vegas.

[5]A version of the FAST implementation deals with the problem of insufficient buffering by choosing α among a small set of pre-determined values based on achieved throughput. This can sometimes lead to unfair throughput allocation as reported in some of the literature. This version was used around early 2004, but discontinued since.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

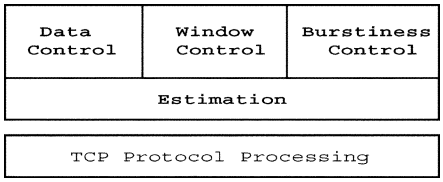

Fig. 1. FAST TCP architecture.

be different from those in the current literature of TCP congestion control. It also reveals some interesting global properties of FAST TCP at a single link.

Corollary 4: Suppose there is only a single link with capacity c.

1) The equilibrium point (w^*, p^*) is given by

$$w_i^* = \alpha_i + \frac{\alpha_i c}{\sum_j \alpha_j} d_i, \quad p = \frac{1}{c} \sum_i \alpha_i$$

with $x_i^* = c\alpha_i / \sum_j \alpha_j$.

2) Starting from any initial point $(w(0), p(0))$, the link is fully utilized, i.e., equality holds in (5), after a finite time.
3) The queue length is lower and upper bounded after a finite time. If all sources have the same propagation delay, $d_i = d$ for all i, then the system converges in finite time.

The stability result reported here is limited to local asymptotic stability at a single link with heterogeneous sources and feedback delay is ignored. In [65], the local stability result is extended to a multilink network in the absence of feedback delay. With feedback delay, local stability can be maintained for the case of a single link, provided the heterogeneity of the delays is small. This delayed stability result is extended in [66] to a multilink network; furthermore, global stability at a single link is established in the absence of delay using a Lyapunov argument. These results are summarized in [67]. In [9], a condition for global asymptotic stability is given for a single-source single-link network.

IV. Performance

We have conducted experiments on our dummynet [55] testbed comparing performance of various new TCP algorithms as well as the Linux TCP implementation. For more complex scenarios that are hard to reliably emulate with our dummynet testbed, we report some simulation results on ns-2 [77]. The experiment and simulation results reported aim to zoom in on specific properties of FAST. These scenarios may be incomplete or unrealistic. Experiments in production networks can be found in [17] and [26]. Other results not presented in this paper are collected in [70] and [79].

A. Testbed and Kernel Instrumentation

Our testbed consists of a sender and a receiver both running Linux, that are connected through an emulated router running dummynet under FreeBSD. Each testbed machine has dual Xeon 2.66 GHz CPUs, 2 GB of main memory, and dual on-board Intel PRO/1000 Gigabit Ethernet interfaces. We have tested these machines to ensure each is able to achieve a peak throughput of 940 Mb/s with the standard Linux TCP protocol using `iperf`. The testbed router supports paths of various delays and a single bottleneck capacity with a fixed buffer size. It has monitoring capability at the sender and the router. The receiver runs multiple `iperf` sinks with different port numbers for connections with different RTTs. We configured dummynet to create paths or pipes of different RTTs, 50, 100, 150, and 200 ms, using different destination port numbers on the receiving machine. We then created another pipe to emulate a bottleneck capacity of 800 Mb/s and a buffer size of 2000 packets, shared by all the delay pipes. To reduce scheduling granularity, we recompiled the FreeBSD kernel so the task scheduler runs every 1 ms. We also increased the size of the IP layer interrupt queue to 3000 to accommodate large bursts of packets. For each connection on the sending machine, the kernel monitor captures the congestion window, the observed base RTT, and the observed queueing delay. On the dummynet router, the kernel monitor captures the throughput at the bottleneck, the number of lost packets, and the average queue size every two seconds.

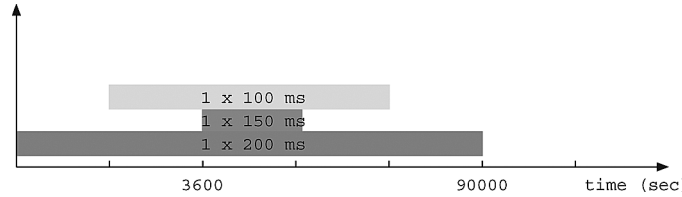

Fig. 2. Dynamic scenario I (3 flows): active periods.

We tested five TCP implementations: FAST, Reno (Linux), HSTCP, STCP, and BIC-TCP using their default parameters for all experiments. The FAST TCP is based on Linux 2.4.20 kernel (α is set to 200 packets), HSTCP, Scalable TCP and Reno are based on Linux 2.4.19 kernel, BIC TCP was based on 2.4.25 kernel. We ran tests and did not observe any appreciable difference among the three plain Linux kernels, and the TCP source codes of the three kernels are nearly identical. Linux TCP implementation includes all of the latest RFCs such as New Reno, SACK, D-SACK, and TCP high performance extensions. There are two versions of HSTCP [10], [38]. We present the results of the implementation in [38], but our tests show that the implementation in [10] has comparable performance.

In all of our experiments, the bottleneck capacity is 800 Mb/s—roughly 67 packets/ms, and the maximum buffer size is 2000 packets.

B. Case Study: Static Scenario I

In the first dynamic test, the number of flows was small so that throughput per flow, and hence the window size, was large. There were three TCP flows, with propagation delays of 100, 150, and 200 ms, that started and terminated at different times, as illustrated in Fig. 2.

For each dynamic experiment, we generated two sets of figures. From the sender monitor, we obtained the trajectory of individual connection throughput (in Kb/s) over time. They are shown in Fig. 3. As new flows joined or old flows left, FAST TCP converged to the new equilibrium rate allocation rapidly and stably (left column). Reno's throughput was also relatively smooth because of the slow (linear) increase between packet losses. It incurred inefficiency towards the end of the experiment when it took 30 minutes for a flow to consume the spare capacity made available by the departure of another flow. HSTCP, STCP, and BIC-TCP responded more quickly but also exhibited significant fluctuation in throughput.

From the queue monitor, we obtained average queue size (packets) shown in Fig. 3 on the right column. The queue

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

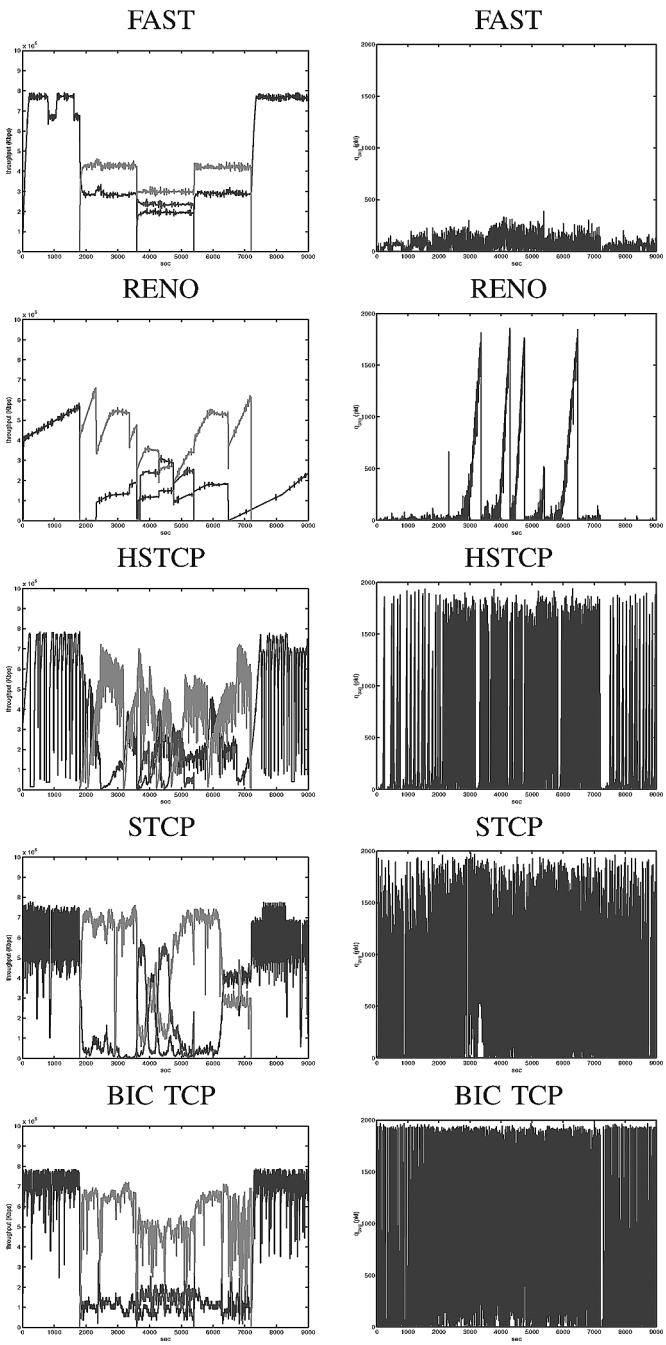

Fig. 3. Dynamic scenario I: throughput trajectory (left column) and Dummynet queue trajectory (right column).

under FAST TCP was quite small throughout the experiment because the number of flows was small. HSTCP, STCP, and BIC-TCP exhibited strong oscillations that filled the buffer. Since BIC-TCP tried to maintain an aggregate window to be just below the point where overflow occurs, it had the highest average queue length.

From the throughput trajectories of each protocol, we calculate Jain's fairness indexes (see Section IV-D for definition) for the rate allocations for each time interval that contains more than one flow (see Fig. 2). The fairness indexes are shown in Table III. FAST TCP obtained the best intra-protocol fairness, very close to 1, followed by HSTCP, Reno, BIC-TCP, and then

TABLE III
DYNAMIC SCENARIO I: INTRA-PROTOCOL FAIRNESS (JAIN'S INDEX)

Time (sec)	#Sources	FAST	Reno	HSTCP	STCP	BIC
1800 – 3600	2	.967	.684	.927	.573	0.683
3600 – 5400	3	.970	.900	.831	.793	0.687
5400 – 7200	2	.967	.718	.873	.877	0.704

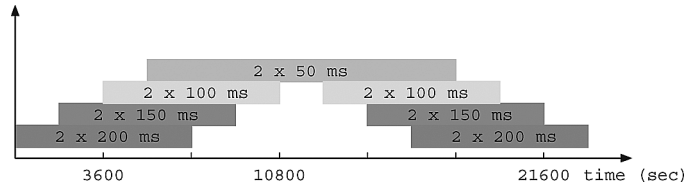

Fig. 4. Dynamic scenario II (8 flows): active periods.

STCP. It confirms that FAST TCP does not penalize flows with large propagation delays.

For FAST TCP, each source tries to maintain the same number of packets in the queue in equilibrium, and thus, in theory, each competing source should get an equal share of the bottleneck bandwidth. Even though FAST TCP achieved the best fairness index, we did not observe the expected equal sharing of bandwidth (see Fig. 3). Our sender monitor showed that all the flows measured their propagation delays correctly. We found that connections with longer RTTs consistently observed higher queueing delays than those with shorter RTTs. For example, the connection on the path of 100 ms saw an average queueing delay of 6 ms, while the connection on the path of 200 ms saw an average queueing delay of 9 ms. This caused the connection with longer RTTs to maintain fewer packets in the queue in equilibrium, thus getting a smaller share of the bandwidth. We conjecture that a larger window size (due to longer RTT) produces a more bursty traffic. With bursty traffic arriving at a queue, each packet would see a delay that includes the transmission times of all preceding packets in the burst, leading to a larger average queueing delay and a smaller throughput.

C. Case Study: Dynamic Scenario II

This experiment was similar to dynamic scenario I, except that there were a larger number (8) of flows, with different propagation delays, which joined and departed according to the schedule in Fig. 4. The qualitative behavior in throughput, fairness, stability, and responsiveness for each of the protocols is similar in this case as in scenario I, and in fact is amplified as the number of flows increases.

Specifically, as the number of competing sources increases, stability became worse for the loss-based protocols. As shown in Fig. 5, oscillations in both throughput and queue size are more severe for loss-base protocols. Packet loss was more severe. The performance of FAST TCP did not degrade in any significant way. Connections sharing the link achieved very similar rates. There was a reasonably stable queue at all times, with little packet loss and high link utilization. Intra-protocol fairness is shown in Table IV, with no significant variation in the fairness of FAST TCP.

D. Overall Evaluation

We have conducted several other experiments, with different delays, number of flows, and their arrival and departure patterns. In all these experiments, the bottleneck link capacity was

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DYNAMIC SCENARIO II: INTRA-PROTOCOL FAIRNESS (JAIN'S INDEX)

Time (sec)	Sources	FAST	Reno	HSTCP	STCP	BIC
0 – 1800	2	1.000	.711	.806	.999	.979
1800 – 3600	4	.987	.979	.940	.721	.971
3600 – 5400	6	.976	.978	.808	.631	.876
5400 – 7200	8	.977	.830	.747	.566	.858
7200 – 9000	6	.970	.845	.800	.613	.856
9000 – 10800	4	.989	.885	.906	.636	.973
10800 – 12600	2	.998	.993	.996	.643	1.000
12600 – 14400	4	.989	.782	.843	.780	.936
14400 – 16200	6	.944	.880	.769	.613	.905
16200 – 18000	8	.973	.787	.816	.547	.779
18000 – 19800	6	.982	.892	.899	.563	.894
19800 – 21600	4	.995	.896	.948	.668	.948
21600 – 23400	2	1.000	1.000	.920	.994	.998

800 Mb/s and buffer size 2000 packets. We present here a summary of protocol performance in terms of some quantitative measures on throughput, fairness, stability, and responsiveness.

We use the output of `iperf` for our quantitative evaluation. Each `iperf` session in our experiments produced five-second averages of its throughput. This is the data rate (i.e., goodput) applications such as `iperf` receives, and is slightly less than the bottleneck bandwidth due to packet header overheads.

Let $x_i(k)$ be the average throughput of flow i in the five-second period k. Most tests involved dynamic scenarios where flows joined and departed. For the definitions below, suppose the composition of flows changes in period $k = 1$, remains fixed over period $k = 1, \ldots, m$, and changes again in period $k = m + 1$, so that $[1, m]$ is the maximum-length interval over which the same equilibrium holds. Suppose there are n active flows in this interval, indexed by $i = 1, \ldots, n$. Let

$$\overline{x}_i := \frac{1}{m} \sum_{k=1}^{m} x_i(k)$$

be the average throughput of flow i over this interval. We now define our performance metrics for this interval $[1, m]$ using these throughput measurements.

1) **Throughput**: The average aggregate throughput for the interval $[1, m]$ is defined as $E := \sum_{i=1}^{n} \overline{x}_i$.
2) **Intra-protocol fairness**: Jain's fairness index for the interval $[1, m]$ is defined as [22] $F := \left(\sum_{i=1}^{n} \overline{x}_i\right)^2 / \left(n \sum_{i=1}^{n} \overline{x}_i^2\right)$. $F \in (0, 1]$ and $F = 1$ is ideal (equal sharing).
3) **Stability**: The stability index of flow i is the sample standard deviation normalized by the average throughput:

$$S_i := \frac{1}{\overline{x}_i} \sqrt{\frac{1}{m-1} \sum_{k=1}^{m} (x_i(k) - \overline{x}_i)^2}.$$

The smaller the stability index, the less oscillation a source experiences. The stability index for interval $[1, m]$ is the average over the n active sources $S := \sum_{i=1}^{n} S_i / n$.

4) **Responsiveness**: The responsiveness index measures the speed of convergence when network equilibrium changes at $k = 1$, i.e., when flows join or depart. Let $\overline{x}_i(k) := \sum_{t=1}^{k} x_i(t)/k$ be the running average by period $k \leq m$. Then $\overline{x}_i(m) = \overline{x}_i$ is the average over the entire interval $[1, m]$.

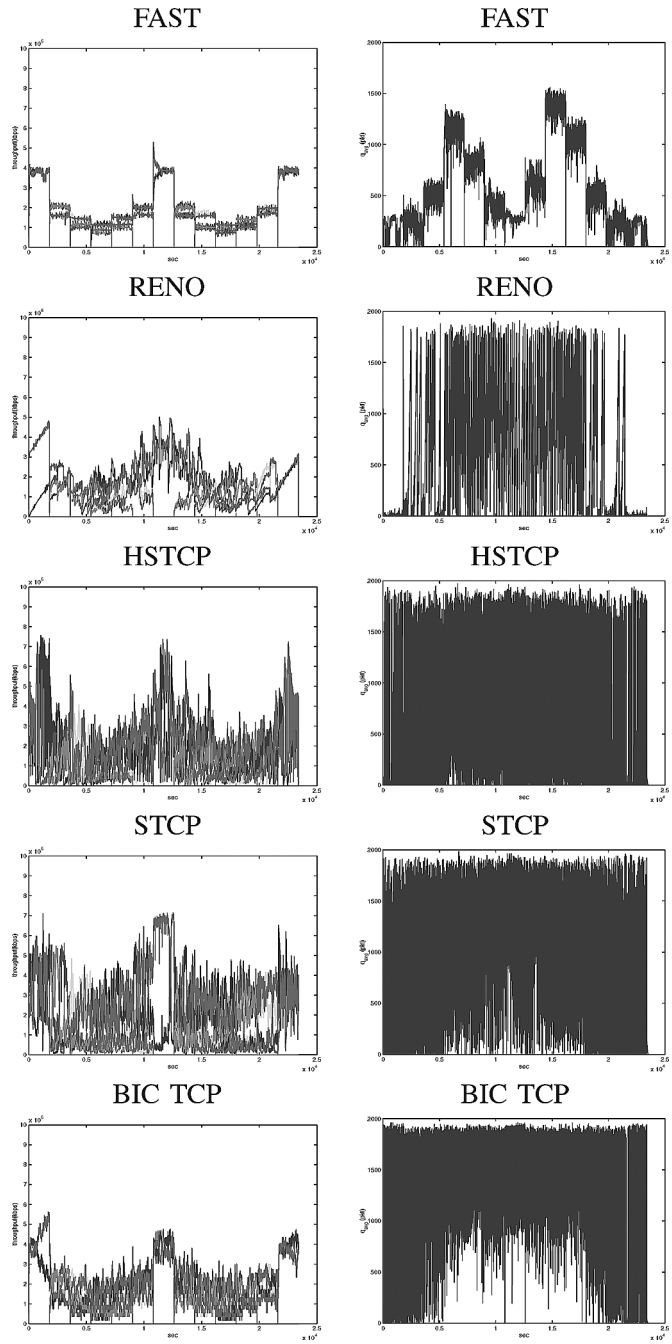

Fig. 5. Dynamic scenario II: throughput trajectory (left column) and Dummynet queue trajectory (right column).

Responsiveness index R_1 measures how fast the running average $\overline{x}_i(k)$ of the *slowest* source converges to $\overline{x}_i$:[6]

$$R_1 := \max_i \max \left\{ k : \left| \frac{\overline{x}_i(k) - \overline{x}_i}{\overline{x}_i} \right| > 0.1 \right\}.$$

[6]The natural definition of responsiveness index as the earliest period after which the throughput $x_i(k)$ (as opposed to the running average $\overline{x}_i(k)$ of the throughput) stays within 10% of its equilibrium value is unsuitable for TCP protocols that do not stabilize into an equilibrium value. Hence, we define it in terms of $\overline{x}_i(k)$ which, by definition, always converges to $\overline{x}_i$ by the end of the interval $k = m$. This definition captures the intuitive notion of responsiveness if $x_i(k)$ settles into a periodic limit cycle.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

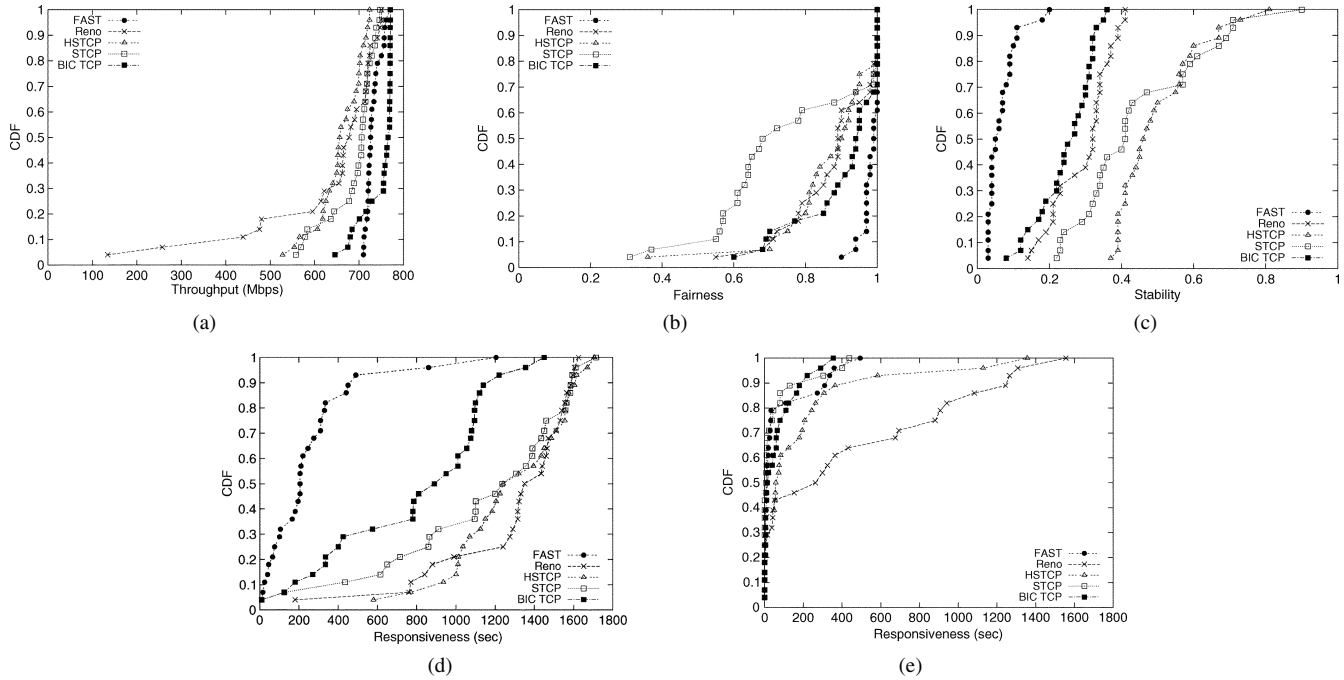

Fig. 6. Overall evaluation: (a) throughput; (b) fairness; (c) stability; (d) responsiveness index R_1; (e) responsiveness index R_2.

Responsiveness index R_2 measures how fast the aggregate throughput converges to $\sum_i \overline{x}_i$:

$$R_2 := \max\left\{k : \left|\frac{\sum_i (\overline{x}_i(k) - \overline{x}_i)}{\sum_i \overline{x}_i}\right| > 0.1\right\}.$$

For each TCP protocol, we obtain one set of computed values for each evaluation criterion for all of our experiments. We plot the cumulative distribution function (CDF) of each set of values. These are shown in Fig. 6(a)–(d).

From Fig. 6(a)–(d), FAST has the best performance among all protocols for three evaluation criteria, fairness, stability and responsiveness index R_1. It has the second best overall throughput. More importantly, the variation in each of the distributions is smaller under FAST than under the other protocols, suggesting that FAST had fairly consistent performance in our test scenarios. We also observed that both HSTCP and STCP achieved higher throughput and improved responsiveness compared with TCP Reno. STCP had worse intra-protocol fairness compared with TCP Reno, while both BIC-TCP and HSTCP achieved comparable intra-protocol fairness to Reno. HSTCP, BIC-TCP, and STCP showed increased oscillations compared with Reno (Fig. 6(c), Fig. 3), and the oscillations became worse as the number of sources increased (Fig. 5).

From Fig. 6(d), FAST TCP achieved a much better responsiveness index R_1 than the other schemes. We caution however that it can be hard to quantify "responsiveness" for protocols that do not stabilize into an equilibrium point or a periodic limit cycle, and hence the unresponsiveness of Reno, HSTCP, and STCP, as measured by index R_1, should be interpreted with care. Indeed, from Fig. 6(e), all protocols except TCP Reno perform well on the responsiveness index R_2 which, unlike R_1, is based on *aggregate* throughput. This apparent discrepancy reflects the fact that link utilization traces converge more quickly than individual throughput traces. It also serves as a justification for the link model (5): the *aggregate* input rate to a link converges more rapidly than individual rates, and hence the queue stabilizes quickly to its new level that tracks changes in windows.

E. ns-2 Simulations

Our dummynet testbed is limited to experiments with single-bottleneck networks and identical protocol. We conducted ns-2 simulation to study the performance of FAST in more complex environments. The FAST implementation in ns-2 is from CUBIN Lab [78]. To eliminate possible simulation artifacts, such as phase effect, we introduced two-way noise traffic in the simulation, where a certain number of Pareto on–off flows with shape parameter 1.5 were introduced in *each* direction.[7] When a noise flow is "on", it transmits at a constant rate of 4 Mb/s. Each noise flow has an average burst time of 100 ms and an average idle time of 100 ms. Hence, the average length of a flow is 50 KB, similar to web traffic. We repeated each scenario 20 times and report both the average rate and the standard deviation (error bars in the figures).

Three sets of simulations were conducted: FAST with different noise levels, FAST with Reno traffic, and FAST on a multilink network. We only present a few examples from each set of simulations; see [79] for complete details.

1) FAST With Noise Traffic: This set of simulations repeated the scenario in Section IV-B, with different levels of noise traffic. The noise traffic consists of Pareto on–off flows as described above. We varied the number of noise flows from 0 to 200, corresponding to an aggregate noise traffic of 0% to 50% of the bottleneck capacity. Fig. 7(a)–(c) show the throughput trajectory of three cases: 0%, 10% (40 noise flows) and 30% (120 noise flows). Each point in the figures represents the average rate over a 60-second interval.

The ns-2 simulation with 0% noise [Fig. 7(a)] should be compared with dummynet experiment in Section IV-B. Different from dummynet experiments, the ns-2 simulation was clean,

[7]We also conducted simulations with exponential on–off traffic. The results are similar.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

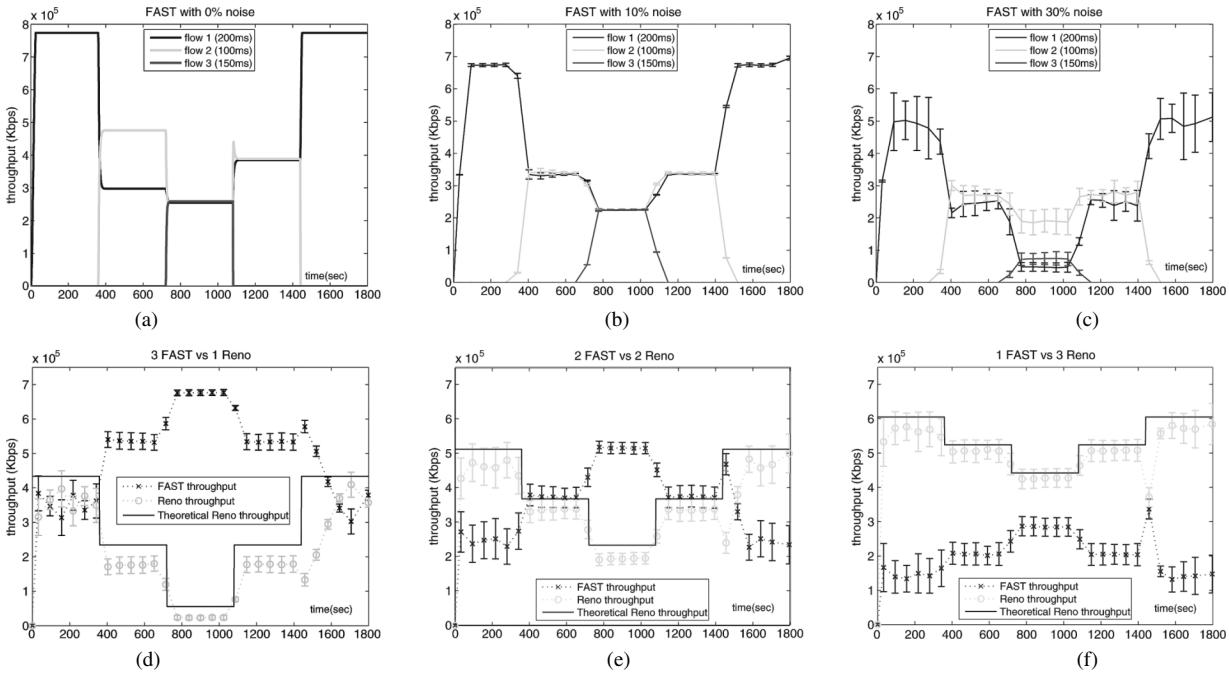

Fig. 7. (a) FAST with 0% mice traffic. (b) FAST with 10% mice traffic. (c) FAST with 30% mice traffic. (d) Three FAST flows versus one Reno flow. (e) Two FAST flows versus two Reno flows. (f) One FAST flow versus three Reno flows.

and new flows mistook queueing delay due to existing flows as part of their propagation delays, leading to unfair throughputs. However, when the noise was 10% of the capacity, such unfairness was eliminated. The queue was frequently emptied and new flows observed the correct propagation delays and converged to the correct equilibrium rates, as shown in Fig. 7(b). Fig. 7(c) shows the throughput when the noise was 30% of the capacity. FAST throughputs oscillated, adapting to mice that joined and left frequently. In the period of 720 to 1080 seconds, the mice traffic generated so much packet loss that the three FAST flows could not keep α packets in the queue and they behaved like an AIMD algorithm. Such AIMD behavior led to discrimination against long RTT flows (flow 1 and flow 3).

2) Interaction Between FAST and Reno: In this set of simulations, we used the same set of paths as in Section IV-B, but we reduced the delay on each path to one-fifth of the original value since Reno took a very long time to reach congestion avoidance with the delays in the original setup. On each path, we used four parallel flows instead of a single flow. We varied the number Reno flows on each path from zero to four (and the number of FAST flows was hence varied from four to zero) to examine FAST's interaction with Reno. The equilibrium rates of FAST and Reno sharing a single bottleneck link are predictable; see [79] for details. Fig. 7(d)–(f) show the aggregate throughputs of FAST flows and Reno flows when the number of Reno flows on each path is 1, 2, and 3. We also present the theoretic predictions on Reno's throughputs on the same figures for comparison. The aggregate throughputs in simulations match the model predictions reasonably well. Reno's throughput is slightly lower than prediction since the model does not captures Reno's timeout behavior. The simulation results also show that FAST may be more aggressive, or less aggressive than Reno, depending on the network setup.

Indeed, the behavior of a *general multilink* network share by heterogeneous protocols that use different congestion signals, such as FAST (which uses delay) and Reno (which uses loss), can be very intricate. See [60] and [61] for details and more comments in Section V.

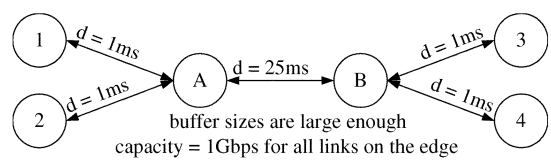

Fig. 8. Topology with multiple bottlenecks.

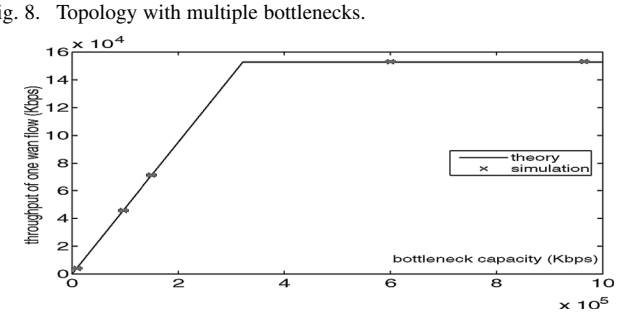

Fig. 9. FAST with multiple bottlenecks.

3) FAST in a Network With Multiple Bottlenecks: We simulated a network with two LANs connected by a WAN. The topology is shown in Fig. 8. Three pairs of flows ran simultaneously on three different paths. The first pair of flows (WAN flows) ran on the path $1 \rightarrow A \rightarrow B \rightarrow 4$. The second pair (LAN flows) ran on $1 \rightarrow A \rightarrow 2$. The third pair (LAN flows) ran on $3 \rightarrow B \rightarrow 4$. All links except $A \rightarrow B$ had a capacity of 1 Gb/s. The capacity of $A \rightarrow B$ was varied from 10 Mb/s to 1 Gb/s. The noise traffic introduced in each link has an average rate of 5% of the link capacity. Link $1 \rightarrow A$ and link $B \rightarrow 4$ were bottlenecks. Link $A \rightarrow B$ also became a bottleneck when its capacity was less than 333 Mb/s. In all scenarios, FAST converged stably to its equilibrium value, fully utilizing $1 \rightarrow A$ and $B \rightarrow 4$. Fig. 9 shows the throughput of the WAN flow $1 \rightarrow A \rightarrow B \rightarrow 4$ with various capacities on $A \rightarrow B$, both

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

measured from simulations and calculated by solving the utility maximization problem in (6). The measured throughputs match the theoretical predictions very well, validating the theoretical model and Theorem 2 on the equilibrium behavior of FAST.

V. Open Issues and Proposed Solutions

FAST is a new delay-based congestion control algorithm. We motivate our design carefully and support it with tentative analysis and experiments. We now summarize some open problems and potential solutions. More practical experience is needed to assess conclusively the seriousness of these problems and the effectiveness of proposed solutions in real networks and applications.

A. Propagation Delay Measurement

Propagation delay (`baseRTT`) is used in the FAST window control algorithm (4). In a clean network, the queueing delay maintained by existing FAST flows may be mistaken as part of the propagation delay by new flows that join later, as shown in ns-2 simulations in [59]. The effect of this estimation error is equivalent to modifying the underlying utility functions to favor new flows over existing flows; see [44] and [59]. Methods to eliminate this error are suggested in [44], [49], and [59].

Propagation delay measurement can be affected by route change from a shorter to a longer path during the lifetime of a connection. Though route change at the timescale of TCP connections is uncommon, mistaking an increase in propagation delay as congestion will reduce the connection's throughput. A solution is proposed in [49] where the propagation delay is estimated by the minimum RTT observed in a certain preceding period, not since the beginning of the connection, so that the estimate tracks route changes.

B. Queueing Delay Measurement

Queueing delay measurement may be affected by the burstiness of the FAST flows themselves, leading to slight unfairness among flows with different RTTs, as shown in Section IV-B. Such error can be greatly reduced by deploying a burstiness control algorithm in the sender, as shown in [71].

Like Vegas, FAST is affected by queueing delay in reverse path, as shown in [4]. There are a number of ways that have been proposed to eliminate the effect of reverse queueing delay for Vegas without the need for additional supports from receivers, that are applicable to FAST. The method in [36] utilizes the TCP timestamp option that is widely implemented in today's TCP stacks. The calculation is correct even when the sender and receiver's clocks have a *constant* offset. If the clock drift is significant, [54] and [62] provide techniques to accurately synchronize the clocks without GPS (global positioning system). If the sender and receiver have different clock resolutions, the sender can estimate the receiver clock period by observing the number of ticks of receiver's clock during a fixed time interval. A different method is proposed in [16] that does not directly measure the queueing delay. Instead, they measure that actual throughput in the forward direction, and use this measurement in place of $W/(d+q)$.

C. Heterogeneous Protocols

It turns out that a network with heterogeneous protocols that react to different congestion signals can behave in a much more intricate way. In particular, we prove theoretically in [61] that there are networks that have multiple equilibrium points, and demonstrate experimentally in [60] this phenomenon using TCP Reno and Vegas/FAST. We also prove in [61] conditions on network parameters that guarantee global uniqueness of network equilibrium.

We show in [60] that any desired inter-protocol fairness are *in principle* achievable by an appropriate choice of FAST parameter, and that intra-protocol fairness among flows within each protocol is unaffected by the presence of the other protocol except for a reduction in effective link capacities. How to design practical distributed algorithms that use only local information to achieve a desired inter-protocol fairness is however an open problem.

Appendix
Proofs

A. Proof of Lemma 1

Fix w.[8] Define $U_i(x_i) = w_i \log x_i - d_i x_i$ and consider the following optimization problem:

$$\max_{x \geq 0} \quad \sum_i U_i(x_i) \quad \text{subject to} \quad Rx \leq c. \tag{9}$$

Since the objective function is strictly concave and the feasible set is compact, there exists a unique optimal solution x^*. Moreover, since R has full row rank, there exists a unique Lagrange multiplier p^* for the dual problem. See, e.g., [42] for details. We claim that p^* is the unique solution of (5) and, for all i,

$$x_i^* = w_i / \left(d_i + q_i^*\right). \tag{10}$$

Now, (10) can be rewritten as, for all i,

$$\sum_l R_{li} p_l^* = q_i^* = \frac{w_i}{x_i^*} - d_i = U_i'\left(x_i^*\right)$$

which is the Karush–Kuhn–Tucker condition for (9). Hence, (10) holds. Then (5) becomes $\sum_i R_{li} x_i^* \leq c_l$, with equality if $p_l^* > 0$. But this is just the complementary slackness condition for (9). ■

B. Proof of Theorem 2

Clearly unique solution x^* for (6) and unique solution p^* for its dual exist, since the utility functions $\alpha_i \log x_i$ are strictly concave and R is full rank (see, e.g., [42]). We need to show that the dual problem of (6) is indeed given by (7). Now the dual objective function is given by [2]

$$\begin{aligned} D(p) &:= \sum_i \max_{x_i \geq 0} \left(\alpha_i \log x_i - x_i \sum_l R_{li} p_l \right) + \sum_l c_l p_l \\ &= \sum_l c_l p_l - \sum_i \alpha_i \log \sum_i R_{li} p_l \\ &\quad + \sum_i \alpha_i (\log \alpha_i - 1). \end{aligned}$$

Since the last term is independent of p, minimizing $D(p)$ over $p \geq 0$ is the same as minimizing (7) over $p \geq 0$. Hence, there exists a unique solution (x^*, p^*) for (6)–(7).

[8]cf. the proof of a similar result in [50].

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

We now show that (x^*, p^*) is the equilibrium point of (4)–(5). In equilibrium, we have $w_i(t+1) = w_i(t) =: w_i$. From (4), the corresponding queueing delays p_l uniquely defined by (5) must be such that the end-to-end queueing delays are strictly positive, i.e., $q_i = \sum_l R_{li} p_l > 0$ for all i even though some p_l can be zero. Then $\alpha_i(w_i, q_i) = \alpha_i$ in equilibrium, and, from (4), we have $q_i = \sum_l R_{li} p_l = \alpha_i / x_i$, where $x_i := w_i/(d_i + q_i)$. But this is the Karush–Kuhn–Tucker condition for (6). Moreover, (5) is the complementary slackness condition. Hence, the equilibrium of (4)–(5) coincides with the optimal solution of (6), (7), i.e., $w = w^*$ and $p = p^*$. ■

C. Proof of Theorem 3

Let N be the number of sources. Let $q(t) = p(t)$ denote the queueing delay at the single link (omitting the subscripts). It is more convenient to work with normalized window

$$y_i(t) := \frac{w_i(t)}{d_i}. \tag{11}$$

Let $Y(t) := \sum_i y_i(t)$ be the aggregate normalized window. Then $q(t) > 0$ if and only if $Y(t) > c$.

The window control algorithm (4) can be expressed in terms of updates on $y(t)$:

$$y_i(t+1) = \left(1 - \frac{\gamma q(t)}{d_i + q(t)}\right) y_i(t) + \gamma \hat{\alpha}_i \tag{12}$$

where $\hat{\alpha}_i := \alpha_i / d_i$. Let $\hat{\alpha} := \sum_i \hat{\alpha}_i$.

We first prove that the queue is lower bounded by a positive constant after a finite time.

Theorem 5:

1) For all $t > c/\gamma\hat{\alpha}$, we have $q(t) > 0$.
2) Moreover, given any $\epsilon > 0$ we have

$$\frac{\hat{\alpha}}{c} \cdot \min_i d_i - \epsilon < q(t) < \frac{\hat{\alpha}}{c} \cdot \max_i d_i + \epsilon$$

for all sufficiently large t.

Proof (Theorem 5): For the first claim, we will prove that the queue will be nonzero at some $t > c/\gamma\hat{\alpha}$, and that once it is nonzero, it stays nonzero.

Suppose $q(t) = 0$. Summing (12) over i, we have $Y(t+1) = Y(t) + \gamma\hat{\alpha}$, i.e., $Y(t)$ grows linearly in time by $\gamma\hat{\alpha}$ in each period. Since $Y(0) \geq 0$, $Y(t) > c$ after at most $c/\gamma\hat{\alpha}$ periods. Hence, there is some $t > c/\gamma\hat{\alpha}$ such that $q(t) > 0$. We now show that $q(t) > 0$ implies $q(t+1) > 0$.

Since $\gamma < 1$, we have from (12)

$$\begin{aligned} y_i(t+1) &\geq \left(1 - \frac{q(t)}{d_i + q(t)}\right) y_i(t) + \gamma\hat{\alpha}_i \\ &= \frac{d_i(t)}{d_i + q(t)} y_i(t) + \gamma\hat{\alpha}_i. \end{aligned}$$

Summing over i gives

$$Y(t+1) \geq \sum_i \frac{d_i(t)}{d_i + q(t)} y_i(t) + \gamma\hat{\alpha}.$$

But $q(t) > 0$ if and only if

$$\sum_i \frac{d_i(t)}{d_i + q(t)} y_i(t) = c. \tag{13}$$

Hence

$$Y(t+1) \geq c + \gamma\hat{\alpha} > c$$

i.e., $q(t+1) > 0$. This proves the first claim.

For the second claim, we first prove that $Y(t)$ converges to its limit point $Y^* := c + \hat{\alpha}$ geometrically (and monotonically):

$$Y(t) = Y^* + (Y(0) - Y^*)(1-\gamma)^t. \tag{14}$$

To prove (14), rewrite (12) as

$$y_i(t+1) = (1-\gamma) y_i(t) + \gamma \left(\frac{d_i y_i(t)}{d_i + q(t)} + \hat{\alpha}_i\right).$$

Summing over i and using (13), we have

$$Y(t+1) = (1-\gamma) Y(t) + \gamma(c + \hat{\alpha})$$

from which (14) follows.

Noting that $d/(d+q)$ is a strictly increasing function of d, we have from (13)

$$\begin{aligned} \frac{\min_i d_i(t)}{\min_i d_i + q(t)} \cdot Y(t) &\leq \sum_i \frac{d_i(t)}{d_i + q(t)} y_i(t) = c \\ &\leq \frac{\max_i d_i(t)}{\max_i d_i + q(t)} \cdot Y(t). \end{aligned}$$

Hence

$$1 + \frac{q(t)}{\min_i d_i} \geq \frac{Y(t)}{c} \geq 1 + \frac{q(t)}{\max_i d_i}. \tag{15}$$

From (14), we have

$$\frac{Y(t)}{c} = 1 + \frac{\hat{\alpha}}{c} + \left(\frac{Y(0) - Y^*}{c}\right)(1-\gamma)^t.$$

Hence, (15) becomes

$$\begin{aligned} \frac{q(t)}{\min_i d_i} &\geq \frac{\hat{\alpha}}{c} + \left[\left(\frac{Y(0) - Y^*}{c}\right)(1-\gamma)^t\right] \\ &\geq \frac{q(t)}{\max_i d_i}. \end{aligned}$$

Since $\gamma \in (0, 1]$, the absolute value of the term in the square bracket can be made arbitrarily small by taking sufficiently large t. Hence, given any $\epsilon' > 0$,

$$\frac{q(t)}{\min_i d_i} \geq \frac{\hat{\alpha}}{c} - \epsilon'$$

and

$$\frac{q(t)}{\max_i d_i} \leq \frac{\hat{\alpha}}{c} + \epsilon'$$

for all sufficiently large t. This proves the second claim.[9] ■

Hence, without loss of generality, we will assume

$$q(t) > \frac{\hat{\alpha}}{2c} \cdot \min_i d_i \qquad \text{for all } t \geq 0.$$

[9]When $\gamma = 1$, then the proof shows that we can set $\epsilon = 0$ in the statement of Theorem 5 after at most $c/\hat{\alpha}$ periods. Moreover, $Y(t) = c + \hat{\alpha}$ for all $t \geq 1$. It also implies that, if $d_i = d$ for all i, then $q(t) = \hat{\alpha} d / c$ for all $t \geq c/\hat{\alpha}$, i.e., the system converges in finite time.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

This implies that, for all $t \geq 0$, equality holds in (5), or equivalently, (13) holds.

More generally, for all $y \in \Re_+^N$ and $q \in \Re_+$, let

$$G(y,q) := \sum_i \frac{d_i y_i}{d_i + q} - c = 0. \tag{16}$$

Lemma 1 guarantees that given any $y \in \Re_+^N$, there is a unique $q \in \Re_+$ that satisfies (16).

An important implication of Theorem 5–2 is that we can restrict our space of y to a subset of $\Re_+^N$:

$$\mathcal{Y} := \Big\{ y \in \Re_+^N \mid \text{the unique } q(y) \text{ defined implicitly by (16) is greater than } \hat{\alpha} \cdot \min_i d_i/(2c) \Big\}. \tag{17}$$

The key feature of $\mathcal{Y}$ we will need in Lemma 9 is that, for all $y \in \mathcal{Y}$, $q(y)$ is lower bounded *uniformly* in y. Define $F : \mathcal{Y} \longrightarrow \mathcal{Y}$ by

$$F_i(y) := \left(1 - \frac{\gamma q(y)}{d_i + q(y)}\right) y_i + \gamma \hat{\alpha}_i \tag{18}$$

where $q(y)$ is implicitly defined by (16). Then the evolution (12) of the normalized window is $y(t+1) = F(y(t))$. Our main result is to show that the iteration F is locally asymptotically stable by proving that the spectral radius of $\partial F/\partial y$ is strictly less than 1 on $\mathcal{Y}$.

Theorem 6: Fix any $\gamma \in (0,1]$. For all $y \in \mathcal{Y}$, the spectral radius of $\partial F/\partial y$ is strictly less than 1.

Theorem 6 implies a neighborhood of the unique fixed point y^* defined by $y^* = F(y^*)$ such that given any initial normalized window $y(0)$ in this neighborhood, $y(t+1) = F(y(t))$ converges to y^*. This implies Theorem 3.

Sketch of proof (Theorem 6): We will show through Lemmas 7–9 that the spectral radius $\rho(\partial F/\partial y)$ is uniformly bounded away from 1, i.e., given $\gamma \in (0,1]$, there exists $\eta' > 0$ such that for all $y \in \mathcal{Y}$,

$$\rho\left(\frac{\partial F}{\partial y}\right) < \eta' < 1. \tag{19}$$

Let $q(y)$ denote the unique solution of (16). Let

$$\beta_i := \frac{d_i y_i}{(d_i + q(y))^2} \left[\sum_j \frac{d_j y_j}{(d_j + q(y))^2} \right]^{-1} \tag{20}$$

$$\mu_i := \frac{d_i}{d_i + q(y)}. \tag{21}$$

By Theorem 5 (2), we have

$$0 < \beta_i, \quad \mu_i < 1 \quad \text{and} \quad \sum_i \beta_i = 1.$$

Let $M := \operatorname{diag}(\mu_i)$ be the diagonal matrix with μ_i as its nonzero entries. Let $\beta := (\beta_i, \text{for all } i)^T$ and $\mu := (\beta_i, \text{ for all } i)^T$ be column vectors.

The proof of the following lemma is straightforward and can be found in [24].

Lemma 7: For $\gamma \in (0,1]$,

$$\frac{\partial F}{\partial y} = \gamma(M - \beta\mu^T) + (1-\gamma)I$$

where I is the $N \times N$ identity matrix.

Let the eigenvalues of $\partial F/\partial y$ be denoted by $\lambda_i(\gamma)$, $i = 1, \ldots, N$, as a function of $\gamma \in (0,1]$. We will show in the next two lemmas that when $\gamma = 1$,

$$0 \leq \lambda_i(1) < 1 \qquad \text{for all } i. \tag{22}$$

Then Lemma 7 implies that, for all $\gamma \in (0,1]$,

$$0 \leq \lambda_i(\gamma) = \gamma\lambda_i(1) + (1-\gamma) < 1 \qquad \text{for all } i$$

and hence (19) holds for any given $\gamma \in (0,1]$.

The key observation to proving (22) is that we can explicitly characterize all the eigenvalues of $\partial F/\partial y$. These eigenvalues are functions of y even though this is not explicit from the notation. Fix $\gamma = 1$ and fix any y. Suppose the set $\{\mu_1, \ldots, \mu_N\}$ takes $k \leq N$ distinct values. Without loss of generality suppose $\mu_1, \ldots, \mu_{j_1}$ take the value $\tilde{\mu}_1, \mu_{j_1+1}, \ldots, \mu_{j_2}$ take the value $\tilde{\mu}_2, \ldots, \mu_{j_{k-1}+1}, \ldots, \mu_{j_k}$ take the value $\tilde{\mu}_k$, such that $\sum_{i=1}^k j_i = N$. The following lemma characterizes completely the eigenvalues and eigenvectors of the Jacobian, and is proved in [24].

Lemma 8: Suppose $\gamma = 1$ and fix any y. Then

1) $\lambda_1 = 0$ is an eigenvalue of $\partial F/\partial y$ with corresponding eigenvector $v_1 = M^{-1}\beta$.
2) For $i = 1, \ldots, k$, if $j_i > 1$ then $\tilde{\mu}_i$ is an eigenvalue with algebraic and geometric multiplicity $j_i - 1$. There are $N-k$ such distinct eigenvalues.
3) The remaining $k-1$ eigenvalues are the solutions of

$$\sum_{i=1}^k \frac{\tilde{\beta}_i}{\tilde{\mu}_i - \lambda} = 0 \tag{23}$$

counting (algebraic) multiplicity, where $\tilde{\beta}_i := \sum_{j=1}^{j_i} \beta_j$. The eigenvectors corresponding to these eigenvalues λ_i, $i = 2, \ldots, k$, are

$$v_i = (M - \lambda_i I)^{-1} \beta_i. \tag{24}$$

The following lemma proves the assertion and is proved in [24].

Lemma 9: Suppose $\gamma = 1$. Then

$$\rho\left(\frac{\partial F}{\partial y}\right) \leq \max_i \mu_i < \max_i \frac{d_i}{d_i + \underline{q}} < 1$$

where $\underline{q} := \hat{\alpha} \min_i d_i / 2c > 0$.

This completes the proof of Theorem 6, from which Theorem 3 follows.

D. Proof of Corollary 4

From (12) and (13), the equilibrium windows $w_i^* = y_i^* d_i$ and delay $q^* = p^*$ satisfy

$$\frac{p^* w_i^*}{d_i + p^*} = \alpha_i \qquad \text{for all } i \tag{25}$$

$$\sum_i \frac{w_i^*}{d_i + p^*} = c. \tag{26}$$

Summing (25) over i and substituting in (26), we have $p^* = \alpha/c$, where $\alpha := \sum_i \alpha_i$. Substituting into (25), we have

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

$$w_i^* = \alpha_i \left(1 + \frac{d_i}{p^*}\right) = \alpha_i + \frac{\alpha_i c}{\alpha} d_i.$$

Hence

$$x_i^* = \frac{w_i^*}{d_i + p^*} = \frac{\alpha_i}{\alpha} c.$$

The second and third claims follow from Theorem 5 and footnote 10 at the end of the proof of Theorem 6. ■

ACKNOWLEDGMENT

The authors gratefully acknowledge the contributions of the FAST project team and their collaborators, at http://www.netlab.caltech.edu/FAST/, in particular, G. Almes, J. Bunn, D. H. Choe, R. L. A. Cottrell, V. Doraiswami, J. C. Doyle, W. Feng, O. Martin, H. Newman, F. Paganini, S. Ravot, S. Shalunov, S. Singh, J. Wang, Z. Wang, and S. Yip, and thank J. Wang for pointing out several errors in an earlier version.

REFERENCES

[1] M. Allman, V. Paxson, and W. Stevens, "TCP Congestion Control," RFC 2581, Apr. 1999.

[2] D. Bertsekas, *Nonlinear Programming*. Belmont, MA: Athena, 1995.

[3] L. S. Brakmo and L. L. Peterson, "TCP Vegas: end-to-end congestion avoidance on a global internet," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 8, pp. 1465–1480, Oct. 1995.

[4] H. Bullot, R. L. Cottrell, and R. Hughes-Jones, "Evaluation of advanced TCP stacks on fast long-distance production networks," *J. Grid Computing*, vol. 1, no. 4, pp. 345–359, Aug. 2004.

[5] C. Casetti, M. Gerla, S. Mascolo, M. Sansadidi, and R. Wang, "TCP Westwood: end-to-end congestion control for wired/wireless networks," *Wireless Netw. J.*, vol. 8, pp. 467–479, 2002.

[6] C. S. Chang and Z. Liu, "A bandwidth sharing theory for a large number of HTTP-like connections," *IEEE/ACM Trans. Netw.*, vol. 12, no. 5, pp. 952–962, Oct. 2004.

[7] D. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Comput. Netw.*, vol. 17, pp. 1–14, 1989.

[8] H. Choe and S. H. Low, "Stabilized Vegas," in *Advances in Communication Control Networks, Lecture Notes in Control and Information Sciences*, S. Tarbouriech, C. Abdallah, and J. Chiasson, Eds. New York: Springer Press, 2004.

[9] J.-Y. Choi, K. Koo, J. S. Lee, and S. H. Low, "Global stability of FAST TCP in single-link single-source network," in *Proc. IEEE Conf. Decision and Control*, Dec. 2005, pp. 1837–1841.

[10] T. Dunigan, Floyd's TCP Slow-Start and AIMD Mods [Online]. Available: http://www.csm.ornl.gov/~dunigan/net100/floyd.html

[11] X. Fan, M. Arcak, and J. T. Wen, "Robustness of network flow control against disturbances and time-delay," *Syst. Contr. Lett.*, vol. 53, no. 11, pp. 13–29, 2004.

[12] W. Feng and S. Vanichpun, "Enabling compatibility between TCP Reno and TCP Vegas," in *IEEE Symp. Applications and the Internet (SAINT 2003)*, Jan. 2003, pp. 301–308.

[13] S. Floyd, M. Handley, J. Padhye, and J. Widmer, "Equation-based congestion control for unicast applications," presented at the ACM SIGCOMM 2000, Stockholm, Sweden.

[14] S. Floyd and T. Henderson, "The NewReno modification to TCP's fast recovery algorithm," RFC 2582, Apr. 1999.

[15] S. Floyd, High speed TCP for large congestion windows. Internet draft draft-floyd-tcp-highspeed-02.txt, work in progress, Feb. 2003 [Online]. Available: http://www.icir.org/floyd/hstcp.html

[16] C. P. Fu and S. C. Liew, "A remedy for performance degradation of TCP Vegas in asymmetric networks," *IEEE Commun. Lett.*, vol. 7, no. 1, pp. 42–44, Jan. 2003.

[17] S. Hegde, D. Lapsley, J. Lindheim, B. Wydrowski, D. Wei, C. Jin, and S. H. Low, "FAST TCP in high-speed networks: an experimental study," presented at the GridNets 2004, San Jose, CA.

[18] J. Hoe, "Improving the startup behavior of a congestion control scheme for TCP," presented at the ACM SIGCOMM'96, Stanford [Online]. Available: http://www.acm.org/sigcomm/sigcomm96/program.html

[19] C. V. Hollot, V. Misra, D. Towsley, and W. B. Gong, "Analysis and design of controllers for AQM routers supporting TCP flows," *IEEE Trans. Autom. Contr.*, vol. 47, no. 6, pp. 945–959, Jun. 2002.

[20] V. Jacobson, "Congestion avoidance and control," presented at the ACM SIGCOMM'88, Stanford, CA [Online]. Available: ftp://ftp.ee.lbl.gov/papers/congavoid.ps.Z

[21] V. Jacobson, R. Braden, and D. Borman, "TCP extensions for high performance," RFC 1323, May 1992.

[22] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation and Modeling*. New York: Wiley, 1991.

[23] ——, "A delay-based approach for congestion avoidance in interconnected heterogeneous computer networks," *ACM Comput. Commun. Rev.*, vol. 19, no. 5, pp. 56–71, Oct. 1989.

[24] C. Jin, D. X. Wei, and S. H. Low, "FAST TCP: Motivation, architecture, algorithms, performance," Caltech, Tech. Rep. CSTR: 2003.010, Dec. 2003 [Online]. Available: http://netlab.caltech.edu/FAST

[25] ——, "TCP FAST: motivation, architecture, algorithms, performance," in *Proc. IEEE INFOCOM*, Mar. 2004, pp. 2490–2501 [Online]. Available: http://netlab.caltech.edu

[26] C. Jin, D. X. Wei, S. H. Low, J. Bunn, D. H. Choe, J. C. Doyle, H. Newman, S. Ravot, S. Singh, F. Paganini, G. Buhrmaster, R. L. A. Cottrell, O. Martin, and W. Feng, "FAST TCP: from theory to experiments," *IEEE Network*, vol. 19, no. 1, pp. 4–11, Jan.-Feb. 2005.

[27] S. Jin, L. Guo, I. Matta, and A. Bestavros, "A spectrum of TCP-friendly window-based congestion control algorithms," *IEEE/ACM Trans. Netw.*, vol. 11, no. 3, pp. 341–355, Jun. 2003.

[28] D. Katabi, M. Handley, and C. Rohrs, "Congestion control for high-bandwidth delay product networks," presented at the ACM SIGCOMM 2002, Pittsburgh, PA [Online]. Available: http://www.ana.lcs.mit.edu/dina/XCP/

[29] F. P. Kelly, "Mathematical modelling of the internet," in *Mathematics Unlimited—2001 and Beyond*, B. Engquist and W. Schmid, Eds. Berlin, Germany: Springer-Verlag, 2001, pp. 685–702.

[30] ——, "Fairness and stability of end-to-end congestion control," *Eur. J. Control*, vol. 9, pp. 159–176, 2003.

[31] F. P. Kelly, A. Maulloo, and D. Tan, "Rate control for communication networks: shadow prices, proportional fairness and stability," *J. Oper. Res. Soc.*, vol. 49, no. 3, pp. 237–252, Mar. 1998.

[32] T. Kelly, "Scalable TCP: improving performance in highspeed wide area networks," *Comput. Commun. Rev.* vol. 32, no. 2, Apr. 2003 [Online]. Available: http://www-lce.eng.cam.ac.uk/ctk21/scalable/

[33] S. Kunniyur and R. Srikant, "Designing AVQ parameters for a general topology network," presented at the Asian Control Conf. Singapore, Sep. 2002.

[34] ——, "A time-scale decomposition approach to adaptive explicit congestion notification (ECN) marking," *IEEE Trans. Autom. Contr.*, vol. 47, no. 6, pp. 882–894, Jun. 2002.

[35] ——, "End-to-end congestion control: utility functions, random losses and ECN marks," *IEEE/ACM Trans. Netw.*, vol. 11, no. 5, pp. 689–702, Oct. 2003.

[36] A. Kuzmanovic and E. Knightly, "TCP-LP: a distributed algorithm for low priority data transfer," in *Proc. IEEE INFOCOM*, 2003, pp. 1691–1701.

[37] T. V. Lakshman and U. Madhow, "The performance of TCP/IP for networks with high bandwidth-delay products and random loss," *IEEE/ACM Trans. Netw.*, vol. 5, no. 3, pp. 336–350, Jun. 1997.

[38] Y. Li, Implementing high-speed TCP. [Online]. Available: http://www.hep.ucl.ac.uk/~ytl/tcpip/hstcp/index.html

[39] S. H. Low, F. Paganini, J. Wang, and J. C. Doyle, "Linear stability of TCP/RED and a scalable control," *Comput. Netw. J.* vol. 43, no. 5, pp. 633–647, 2003 [Online]. Available: http://netlab.caltech.edu

[40] S. H. Low and R. Srikant, "A mathematical framework for designing a low-loss, low-delay internet," *Networks and Spatial Economics, Special Issue on Crossovers Between Transportation Planning and Telecommunications*, vol. 4, pp. 75–101, Mar. 2004.

[41] S. H. Low, "A duality model of TCP and queue management algorithms," *IEEE/ACM Trans. Netw.* vol. 11, pp. 525–536, Aug. 2003.

[42] S. H. Low and D. E. Lapsley, "Optimization flow control, I: basic algorithm and convergence," *IEEE/ACM Trans. Netw.* vol. 7, no. 6, pp. 861–874, Dec. 1999 [Online]. Available: http://netlab.caltech.edu

[43] S. H. Low, F. Paganini, and J. C. Doyle, "Internet congestion control," *IEEE Contr. Syst. Mag.*, vol. 22, no. 1, pp. 28–43, Feb. 2002.

[44] S. H. Low, L. Peterson, and L. Wang, "Understanding Vegas: a duality model," *J. ACM*, vol. 49, no. 2, pp. 207–235, Mar. 2002 [Online]. Available: http://netlab.caltech.edu

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.

[45] J. Martin, A. Nilsson, and I. Rhee, "Delay-based congestion avoidance for TCP," *IEEE/ACM Trans. Netw.*, vol. 11, no. 3, pp. 356–369, Jun. 2003.
[46] L. Massoulie and J. Roberts, "Bandwidth sharing: objectives and algorithms," *IEEE/ACM Trans. Netw.*, vol. 10, no. 3, pp. 320–328, Jun. 2002.
[47] M. Mathis, J. Mahdavi, S. Floyd, and A. Romanow, "TCP selective acknowledgment options," RFC 2018, Oct. 1996.
[48] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM Comput. Commun. Rev.*, vol. 27, no. 3, Jul. 1997 [Online]. Available: http://www.psc.edu/networking/papers/model_ccr97.ps
[49] J. Mo, R. La, V. Anantharam, and J. Walrand, "Analysis and comparison of TCP Reno and Vegas," in *Proc. IEEE INFOCOM'99*, pp. 1556–1563.
[50] J. Mo and J. Walrand, "Fair end-to-end window-based congestion control," *IEEE/ACM Trans. Netw.*, vol. 8, no. 5, pp. 556–567, Oct. 2000.
[51] F. Paganini, J. C. Doyle, and S. H. Low, "Scalable laws for stable network congestion control," in *Proc. Conf. Decision and Control*, Dec. 2001 [Online]. Available: http://www.ee.ucla.edu/~paganini
[52] F. Paganini, Z. Wang, J. C. Doyle, and S. H. Low, "Congestion control for high performance, stability and fairness in general networks," *IEEE/ACM Trans. Netw.*, vol. 13, no. 1, pp. 43–56, Feb. 2005.
[53] F. Paganini, Z. Wang, S. H. Low, and J. C. Doyle, "A new TCP/AQM for stable operation in fast networks," in *Proc. IEEE INFOCOM 2003*, pp. 96–105 [Online]. Available: http://www.ee.ucla.edu/~paganini
[54] A. Pásztor and D. Veitch, "PC based precision timing without GPS," presented at the ACM SIGMETRICS 2002 Marina Del Rey, CA.
[55] L. Rizzo, Dummynet. [Online]. Available: http://info.iet.unipi.it/~luigi/ip_dummynet/
[56] R. Shorten, D. Leith, J. Foy, and R. Kilduff, "Analysis and design of congestion control in synchronised communication networks," in *Proc. 12th Yale Workshop on Adaptive and Learning Systems*, Yale, Univ., New Haven, CT, May 2003 [Online]. Available: http://www.hamilton.ie/doug_leith.htm
[57] R. Srikant, *The Mathematics of Internet Congestion Control*. Cambridge, MA: Birkhauser, 2004.
[58] W. Stevens, "TCP Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery Algorithms," RFC 2001, Jan. 1997.
[59] L. Tan, C. Yuan, and M. Zukerman, "FAST TCP: fairness and queuing issues," *IEEE Commun. Lett.*, vol. 9, no. 8, pp. 762–764, Aug. 2005.
[60] A. Tang, J. Wang, S. Hegde, and S. H. Low, "Equilibrium and fairness of networks shared by TCP Reno and FAST," *Telecommun. Syst.*, vol. 30, no. 4, pp. 417–439, Dec. 2005.
[61] A. Tang, J. Wang, S. H. Low, and M. Chiang, "Equilibrium of heterogeneous congestion control protocols," in *Proc. IEEE INFOCOM 2005*, pp. 1338–1349.
[62] D. Veitch, S. Babu, and A. Pásztor, "Robust remote synchronisation of a new clock for PCs," presented at the Internet Measurement Conf., Taormina, Italy, Oct. 2004.
[63] G. Vinnicombe, "On the stability of networks operating TCP-like congestion control," in *Proc. IFAC World Congress on Automatic Control*, Barcelona, Spain, 2002.
[64] ——, "Robust congestion control for the Internet", Tech. Rep., Univ. Cambridge, 2002.
[65] J. Wang, A. Tang, and S. H. Low, "Local stability of FAST TCP," in *Proc. IEEE Conf. Decision and Control*, Dec. 2004, pp. 1023–1028.
[66] J. Wang, D. X. Wei, and S. H. Low, "Modeling and stability of FAST TCP," in *Proc. IEEE INFOCOM 2005*, pp. 938–948.
[67] J. Wang, D. X. Wei, and S. H. Low, "Modeling and stability of FAST TCP," in *IMA Volumes in Mathematics and its Applications, Vol. 143, Wireless Communications*, P. Agrawal, M. Andrews, P. J. Fleming, G. Yin, and L. Zhang, Eds. New York: Springer Science, 2006.
[68] R. Wang, M. Valla, M. Sanadidi, B. Ng, and M. Gerla, "Using adaptive rate estimation to provide enhanced and robust transport over heterogeneous networks," in *Proc. IEEE ICNP 2002*, pp. 206–215.
[69] Z. Wang and J. Crowcroft, "Eliminating periodic packet losses in the 4.3-Tahoe BSD TCP congestion control algorithm," *ACM Comput. Commun. Rev.*, vol. 22, no. 2, pp. 9–16, Apr. 1992.
[70] D. X. Wei, "Congestion control algorithms for high speed long distance TCP connections" Masters thesis, California Inst. Technol., Pasadena, CA, Jun. 2004 [Online]. Available: http://netlab.caltech.edu/pub/projects/FAST/msthesis-dwei
[71] D. X. Wei, S. H. Low, and S. Hegde, "A burstiness control for TCP," presented at the Workshop on Protocols for Fast Long-Distance Networks (PFLDnet'2005) Lyon, France, Feb. 2005.
[72] E. Weigle and W. Feng, "A case for TCP Vegas in high-performance computational grids," in *Proc. 9th Int. Symp. High Performance Distributed Computing (HPDC'01)*, Aug. 2001, pp. 158–167.
[73] J. T. Wen and M. Arcak, "A unifying passivity framework for network flow control," *IEEE Trans. Autom. Contr.*, vol. 49, no. 2, pp. 162–174, Feb. 2004.
[74] B. Wydrowski, High-resolution one-way delay measurement using RFC1323, Preprint, Aug. 2004.
[75] L. Xu, K. Harfoush, and I. Rhee, "Binary increase congestion control (BIC) for fast long-distance networks," in *Proc. IEEE INFOCOM 2004*, pp. 2514–2524.
[76] H. Yaiche, R. R. Mazumdar, and C. Rosenberg, "A game theoretic framework for bandwidth allocation and pricing in broadband networks," *IEEE/ACM Trans. Netw.*, vol. 8, no. 5, pp. 667–678, Oct. 2000.
[77] The Network Simulator—ns-2. [Online]. Available: http://www.isi.edu/nsnam/ns/
[78] FAST TCP Simulator Module for ns-2. CUBIN Lab [Online]. Available: http://www.cubinlab.ee.mu.oz.au/ns2fasttcp/
[79] Caltech ns-2 Simulation Results of FAST. NetLab [Online]. Available: http://netlab.caltech.edu/pub/projects/FAST/ns2-test

David X. Wei (S'03) received the B.E. degree from Tsinghua University, China, and the M.S. degree from the California Institute of Technology (Caltech), Pasadena, both in computer science. He is currently working toward the Ph.D. degree at Caltech.

Mr. Wei has been a student member of the ACM since 2000.

Cheng Jin received the B.S. degree in electrical engineering from Case Western Reserve University, Cleveland, OH, and the Ph.D. degree in computer science and engineering from the University of Michigan, Ann Arbor. He was a postdoc at Caltech.

Steven H. Low (M'92–SM'99) received the B.S. degree from Cornell University, Ithaca, NY, and the Ph.D. degree from the University of California at Berkeley.

He is a Professor of the Computer Science and Electrical Engineering Departments at Caltech, Pasadena.

Dr. Low was a co-recipient of the IEEE William R. Bennett Prize Paper Award in 1997 and the 1996 R&D 100 Award. He was on the editorial board of IEEE/ACM TRANSACTIONS ON NETWORKING from 1997 to 2006 and of *Computer Networks Journal* from 2003 to 2005. He is on the editorial boards of *ACM Computing Surveys*, *NOW Foundations* and *Trends in Networking*. He is a Senior Editor of the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS and a Co-Editor of the Springer Book Series on Optimization and Control of Communication Systems: Theory and Applications. He is a member of the Networking and Information Technology Technical Advisory Group for the U.S. President's Council of Advisors on Science and Technology (PCAST).

Sanjay Hegde received the M.S. degree from Illinois Institute of Technology, Chicago.

He was a Research Engineer at the Networking Laboratory, Caltech, and is now with Akamai Technologies.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 03,2026 at 18:38:31 UTC from IEEE Xplore. Restrictions apply.